

Designing a Simple OS Task Manager

Overview

This report simulates how an operating system manages tasks and resources. It covers CPU scheduling, safe process synchronization, memory page replacement, and disk access optimization.

Part 1: Process Scheduling

Given Processes

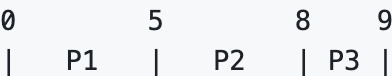
Process	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	1

FCFS Scheduling

First-Come, First-Served runs processes in the order they arrive.

Execution order: P1 -> P2 -> P3

Gantt chart:



Process	Arrival	Burst	Start	Completion	Waiting Time
P1	0	5	0	5	0
P2	1	3	5	8	4
P3	2	1	8	9	6

Average waiting time:

$(0 + 4 + 6) / 3 = 10 / 3 = 3.33$ time units

Round Robin Scheduling, Quantum = 2

Round Robin gives each ready process up to 2 time units before moving it to the back of the queue if it is not finished.

Execution order: P1 -> P2 -> P3 -> P1 -> P2 -> P1

Gantt chart:

```
0    2    4    5    7    8    9
| P1 | P2 | P3 | P1 | P2 | P1 |
```

Process	Arrival	Burst	Completion	Turnaround Time	Waiting Time
P1	0	5	9	9	4
P2	1	3	8	7	4
P3	2	1	5	3	2

Average waiting time:

$(4 + 4 + 2) / 3 = 10 / 3 = 3.33$ time units

Responsiveness Comparison

Both algorithms have the same average waiting time in this scenario: 3.33 time units.

However, Round Robin is more responsive because each process gets CPU access earlier:

Process	FCFS First Response	Round Robin First Response
P1	0	0
P2	4	1
P3	6	2

Round Robin is better for interactive systems because short or newly arrived jobs do not wait for one long process to finish completely.

Part 2: Process Synchronization

Problem Without Synchronization

Two processes, P1 and P2, both increment the shared variable `counter` 100 times. The

expected final value is:

```
100 + 100 = 200
```

Without synchronization, a race condition can happen. Incrementing a counter is not one single CPU operation. It usually means:

1. Read `counter` .
2. Add 1.
3. Write the result back.

If P1 and P2 read the same value before either writes back, one increment can be lost. The final counter may be less than 200.

Synchronization Mechanism: Mutex

A mutex allows only one process at a time to enter the critical section where `counter` is modified.

Pseudocode:

```
counter = 0
mutex = new Mutex()

process P1:
  repeat 100 times:
    mutex.lock()
    counter = counter + 1
    mutex.unlock()

process P2:
  repeat 100 times:
    mutex.lock()
    counter = counter + 1
    mutex.unlock()
```

The `lock()` operation prevents another process from entering the critical section until `unlock()` is called. This enforces mutual exclusion and prevents lost updates.

Part 3: Memory Management

Given Data

Page frames: 3
Reference string: 1, 2, 3, 2, 4, 1, 5

FIFO Page Replacement

FIFO removes the page that entered memory first.

Reference	Frames After Access	Hit/Fault	Removed
1	1, -, -	Fault	-
2	1, 2, -	Fault	-
3	1, 2, 3	Fault	-
2	1, 2, 3	Hit	-
4	2, 3, 4	Fault	1
1	3, 4, 1	Fault	2
5	4, 1, 5	Fault	3

Total FIFO page faults:

6

LRU Page Replacement

LRU removes the page that has not been used for the longest time.

Reference	Frames After Access	Hit/Fault	Removed
1	1, -, -	Fault	-
2	1, 2, -	Fault	-
3	1, 2, 3	Fault	-
2	1, 2, 3	Hit	-
4	2, 3, 4	Fault	1
1	2, 4, 1	Fault	3
5	4, 1, 5	Fault	2

Total LRU page faults:

6

Memory Management Comparison

For this reference string, FIFO and LRU both produce 6 page faults. Neither performs better in this exact case.

LRU is usually more effective in real systems because it uses recent access history. In this small example, the access pattern does not give LRU an advantage.

Part 4: Disk Scheduling

Given Data

Initial disk head position: 53
Requests: 98, 183, 37, 122, 14, 124, 65, 67

FCFS Disk Scheduling

FCFS handles requests in the order they arrive.

Path:

53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67

Move	Distance
53 -> 98	45
98 -> 183	85
183 -> 37	146
37 -> 122	85
122 -> 14	108
14 -> 124	110
124 -> 65	59
65 -> 67	2

Total FCFS head movement:

$45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640$ tracks

CCFS Disk Scheduling

SSTF Disk Scheduling

Shortest Seek Time First chooses the pending request closest to the current head position.

Path:

53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183

Move	Distance
53 -> 65	12
65 -> 67	2
67 -> 37	30
37 -> 14	23
14 -> 98	84
98 -> 122	24
122 -> 124	2
124 -> 183	59

Total SSTF head movement:

12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = 236 tracks

Disk Scheduling Comparison

SSTF is more efficient in this scenario because it reduces total head movement:

Algorithm	Total Head Movement
FCFS	640 tracks
SSTF	236 tracks

SSTF saves:

640 - 236 = 404 tracks

SSTF is more efficient because it always chooses the closest pending request, reducing unnecessary disk head travel.

Final Summary

Topic	Result
FCFS average waiting time	3.33 time units
Round Robin average waiting time	3.33 time units
More responsive CPU algorithm	Round Robin
Synchronization mechanism	Mutex
FIFO page faults	6
LRU page faults	6
Better page replacement here	Tie
FCFS disk head movement	640 tracks
SSTF disk head movement	236 tracks
More efficient disk algorithm	SSTF